

Appunti Tirocinio

Alessio Marini

Appunti presi per il mio Tirocinio interno presso il Vision-Lab.

July 29, 2026

1. Linear Algebra

1.1. Scalars

I numeri che utilizziamo tutti i giorni prendono il nome di **scalari**, li indicheremo con lettere *lower-case* come x, y, z e lo spazio di tutti gli scalari **reali** $\mathbb{R}$.

Implementiamo gli scalari come dei tensor da un solo elemento, vediamo un esempio di assegnamento e delle operazioni:

```
x = torch.tensor(3.0)
y = torch.tensor(2.0)

x + y, x * y, x / y, x**y
# Output
# (tensor(5.), tensor(6.), tensor(1.5000), tensor(9.))
```

1.2. Vectors

Possiamo vedere i vettori come degli array di scalari con lunghezza fissa, gli scalari saranno gli *elementi* del vettore. Quando li utilizzeremo per rappresentare problemi reali avremo che ogni elemento indica un parametro di cosa stiamo analizzando. Indichiamo i vettore con lettere lowercase bold come $\mathbf{x}, \mathbf{y}, \mathbf{z}$.

Ci riferiamo agli elementi di un vettore con la lettera del vettore e l'indice dell'elemento in basse, x_2 indica l'elemento con indice 2 nel vettore $\mathbf{x}$. Normalmente li visualizziamo impilando gli elementi verticalmente:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_3 \end{bmatrix}$$

Per indicare che un vettore contiene n elementi scriviamo che $\mathbf{x} \in \mathbb{R}^n$, formalmente diciamo che n è la **dimensione** dell'array, questa nel codice corrisponde appunto alla lunghezza del tensor che possiamo richiamare con `len(x)` oppure tramite l'attributo `x.shape`.

Per fare chiarezza indichiamo con:

- **Order**: Numero degli assi di un vettore
- **Dimensionality**: Numeri dei componenti

1.3. Matrici

Gli scalari sono tensor di ordine 0, i vettori sono tensor di ordine 1 e le matrici sono di ordine 2. Le indichiamo con lettere upper case bold, quindi $\mathbf{X}, \mathbf{Y}, \mathbf{Z}$ e le rappresentiamo nel codice con vettori con due assi. Con l'espressione $\mathbf{A} \in \mathbb{R}^{m \times n}$ indichiamo che una matrice $\mathbf{A}$ contiene $m \times n$ valori reali ordinati in m righe e n colonne. Quando $m = n$ diciamo che la matrice è *quadrata*. Possiamo illustrarla come una tabella e per indicare un elemento usiamo il doppio pedice, quindi $a_{i,j}$ indica l'elemento nella i -esima riga e alla j -esima colonna di $\mathbf{A}$.

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}$$

É possibile invertire gli assi ovvero scambiare righe con colonne, il risultato é la **matrice trasposta**, la indichiamo con $\mathbf{A}^\top$. Otteniamo che se $\mathbf{B} = \mathbf{A}^\top$ allora $b_{ij} = a_{ji}$ per ogni i, j . Inoltre la trasposta di una matrice $m \times n$ sar  una matrice $n \times m$.

Le matrici **simmetriche** sono un sottoinsieme delle matrici quadrate, sono matrici uguali alla loro trasposta, quindi $\mathbf{A} = \mathbf{A}^\top$

1.4. Tensors

I tensors ci danno modo di rappresentare array di ordine n . Li indichiamo con lettere maiuscole X,Y,Z e per gli elementi utilizziamo lo stesso meccanismo di indici delle matrici. I tensors diventeranno pi  importanti quando lavoreremo con le immagini, ognuna infatti é rappresentata da un tensor di ordine 3 dove gli assi corrispondono a *altezza*, *larghezza* e *canali*. Una collezione di immagini sar  quindi un tensor di ordine 4.

1.4.1. Basic Properties of Tensor Arithmetic

Scalari, vettori, matrici e tensors hanno delle propriet , ad esempio le operazioni *element-wise* producono output della stessa forma degli operandi.

```
A = torch.arange(6, dtype=torch.float32).reshape(2, 3)
B = A.clone() # Alloca nuova memoria per B
A, A + B

# Output
# (tensor([[0., 1., 2.],
#          [3., 4., 5.]]),
#  tensor([[ 0.,  2.,  4.],
#          [ 6.,  8., 10.]])
```

Il prodotto *elementwise* di due matrici si chiama **Hadamard Product**, lo indichiamo con $\odot$. Gli elementi saranno quindi:

$$\mathbf{A} \odot \mathbf{B} = \begin{bmatrix} a_{11}b_{11} & a_{12}b_{12} & \dots & a_{1n}b_{1n} \\ a_{21}b_{21} & a_{22}b_{22} & \dots & a_{2n}b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1}b_{m1} & a_{m2}b_{m2} & \dots & a_{mn}b_{mn} \end{bmatrix}$$

Andando invece a sommare o moltiplicare un tensor con uno scalare otteniamo un tensor con la stessa forma di quello originale ma su ogni elemento viene applicata l'operazione specificata.

1.4.2. Reduction

Spesso ci capiter  di dover calcolare la somma di tutti gli elementi di un tensor. Matematicamente possiamo esprimerla come $\sum_{i=1}^n x_i$ ma nel codice esiste una funzione dedicata:

```
x = torch.arange(3, dtype=torch.float32)
x, x.sum()
```

```
# Output
# (tensor([0., 1., 2.]), tensor(3.))
```

La funzione `sum()` va bene anche per tensors di ordine maggiore. Di base chiamare la funzione somma, *riduce* il tensor su tutti i suoi assi producendo, in alcuni casi, uno scalare. Possiamo specificare su quali assi effettuare la somma e quell'asse ovviamente sparirà dalla dimensione del tensor:

```
A.shape, A.sum(axis=0).shape
# Output
# (torch.Size([2, 3]), torch.Size([3]))

A.shape, A.sum(axis=1).shape
# Output
# (torch.Size([2, 3]), torch.Size([2]))
```

In alcuni casi però potrebbe essere utile si calcolare la somma, ma anche mantenere intatto il tensor, in questi casi impostiamo:

```
sum_A = A.sum(axis=1, keepdims=True)
sum_A, sum_A.shape

# Output
# (tensor([[ 3.],
#          [12.]]),
# torch.Size([2, 1]))
```

1.4.3. Dot Products

È una delle operazioni principali, dati due vettori $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$ il loro *dot product* è dato da $\mathbf{x}^\top \mathbf{y}$, ovvero la somma dei prodotti degli elementi nella stessa posizione:

$$\mathbf{x}^\top \mathbf{y} = \sum_{i=1}^d x_i y_i$$

```
y = torch.ones(3, dtype = torch.float32)
x, y, torch.dot(x, y)

# Output
# (tensor([0., 1., 2.]), tensor([1., 1., 1.]), tensor(3.))
```

In modo perfettamente equivalente possiamo calcolare un dot product effettuando prima un prodotto elementwise e poi una somma.

I dot product sono fondamentali in questo ambito. Ad esempio dato un set di valori in un vettore $\mathbf{x} \in \mathbb{R}^n$ e un set di pesi $\mathbf{w} \in \mathbb{R}^n$; la *weighted sum* di $\mathbf{x}$ in base ai pesi $\mathbf{w}$ può essere espressa come il dot product $\mathbf{x}^\top \mathbf{w}$.

Quando i pesi sono non negati e la loro somma è uguale a 1, il dot product rappresenta una *weighted average*. Inoltre, dopo aver normalizzato due vettori in modo da avere una lunghezza unitaria (pari a 1) il dot product esprimerà il coseno dell'angolo compreso fra loro.

1.4.4. Matrix - Vector Products

Adesso possiamo vedere il prodotto fra una matrice $m \times n$ $\mathbf{A}$ e un vettore n -dimensionale $\mathbf{x}$. Per prima cosa visualizziamo la matrice come vettori riga:

$$\mathbf{A} = \begin{bmatrix} \mathbf{a}_1^\top \\ \mathbf{a}_2^\top \\ \vdots \\ \mathbf{a}_m^\top \end{bmatrix}$$

Dove ogni $\mathbf{a}_i^\top \in \mathbb{R}^n$ é un vettore-riga che rappresenta la i -esima riga di $\mathbf{A}$.

Il prodotto matrice-vettore $\mathbf{Ax}$ é semplicemente un vettore colonna di lunghezza m dove l'elemento i é il dot product $\mathbf{a}_i^\top \mathbf{x}$:

$$\mathbf{Ax} = \begin{bmatrix} \mathbf{a}_1^\top \\ \mathbf{a}_2^\top \\ \vdots \\ \mathbf{a}_m^\top \end{bmatrix} \mathbf{x} = \begin{bmatrix} \mathbf{a}_1^\top \mathbf{x} \\ \mathbf{a}_2^\top \mathbf{x} \\ \vdots \\ \mathbf{a}_m^\top \mathbf{x} \end{bmatrix}$$

Possiamo vedere quindi la moltiplicazione con una matrice $\mathbf{A} \in \mathbb{R}^{m \times n}$ come una trasformazione che porta i vettori da $\mathbb{R}^n$ a $\mathbb{R}^m$. Queste trasformazioni sono molto utili infatti sono l'operazione principale usata in ogni layer dove, dato l'output del layer precedente, viene calcolato l'output attuale.

Per rappresentare questi prodotti nel codice usiamo la funzione `mv`. Per effettuarla é importante che la dimensione di $\mathbf{A}$ sia la stessa di $\mathbf{x}$. In Python inoltre abbiamo un operatore pensato sia per prodotti matrix-matrix che matrix-vector che é `@`:

```
A.shape, x.shape, torch.mv(A, x), A@x

# Output
# (torch.Size([2, 3]), torch.Size([3]), tensor([ 5., 14.]), tensor([ 5., 14.]))
```

1.4.5. Matrix - Matrix Multiplication

Il concetto é molto simile a quello dei dot product. Prendiamo due matrici $\mathbf{A} \in \mathbb{R}^{n \times k}$ e $\mathbf{B} \in \mathbb{R}^{k \times m}$:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1k} \\ a_{21} & a_{22} & \dots & a_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nk} \end{bmatrix}, \mathbf{B} = \begin{bmatrix} b_{11} & b_{12} & \dots & b_{1m} \\ b_{21} & b_{22} & \dots & b_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{k1} & a_{k2} & \dots & a_{km} \end{bmatrix}$$

Sia $\mathbf{a}_i^\top \in \mathbb{R}^k$ il vettore-riga che rappresenta la i -esima riga della matrice $\mathbf{A}$ e sia $\mathbf{b}_j \in \mathbb{R}^k$ il vettore-colonna che rappresenta la j -esima colonna della matrice $\mathbf{B}$:

$$\mathbf{A} = \begin{bmatrix} \mathbf{a}_1^\top \\ \mathbf{a}_2^\top \\ \vdots \\ \mathbf{a}_n^\top \end{bmatrix}, \mathbf{B} = [\mathbf{b}_1 \ \mathbf{b}_2 \ \dots \ \mathbf{b}_m]$$

Per formare la matrice $\mathbf{C} \in \mathbb{R}^{n \times m}$ calcoliamo ogni elemento c_{ij} come il dot product tra la i -esima riga di $\mathbf{A}$ e la j -esima colonna di $\mathbf{B}$:

$$C = AB = \begin{bmatrix} \mathbf{a}_1^\top \\ \mathbf{a}_2^\top \\ \vdots \\ \mathbf{a}_n^\top \end{bmatrix} [\mathbf{b}_1 \ \mathbf{b}_2 \ \dots \ \mathbf{b}_m] = \begin{bmatrix} \mathbf{a}_1^\top \mathbf{b}_1 & \mathbf{a}_1^\top \mathbf{b}_2 & \dots & \mathbf{a}_1^\top \mathbf{b}_m \\ \mathbf{a}_2^\top \mathbf{b}_1 & \mathbf{a}_2^\top \mathbf{b}_2 & \dots & \mathbf{a}_2^\top \mathbf{b}_m \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{a}_n^\top \mathbf{b}_1 & \mathbf{a}_n^\top \mathbf{b}_2 & \dots & \mathbf{a}_n^\top \mathbf{b}_m \end{bmatrix}$$

Possiamo pensare ad una moltiplicazione matrix-matrix AB equivalente ad effettuare m prodotti matrix-vector o $m \times n$ dot products per mettere insieme i risultati e ottenere una matrice $n \times m$.

1.5. Norms

Di base la *norm* di un vettore ci dice «quanto é grande». Ne esistono diverse, ad esempio la l_2 misura la lunghezza Euclidea di un vettore.

La norm é una funzione $\|\cdot\|$ che mappa un vettore in uno scalare e soddisfa le 3 seguenti proprietà:

1. Dato un array $\mathbf{x}$, se scaliamo tutti i suoi elementi per uno scalare $a \in \mathbb{R}$ anche la norm scala:

$$\|a\mathbf{x}\| = |a| \|\mathbf{x}\|$$

2. Dati due vettori $\mathbf{x}$ e $\mathbf{y}$, la norm soddisfa la *triangle inequality*:

$$\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$$

3. La norm di un vettore é non-negativa e si annulla soltanto se il vettore é zero:

$$\|\mathbf{x}\| > 0 \text{ for all } \mathbf{x} \neq 0$$

Esistono diverse funzioni valide come norms e diverse norms rappresentano diversi concetti di grandezza. La norm Euclidea corrisponde alla radice quadrata della somma dei quadrati di un vettore, questa é chiamata norm l_2 e possiamo esprimerla come:

$$\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$$

Nel codice possiamo chiamare semplicemente il metodo `norm`:

```
u = torch.tensor([3.0, -4.0])
torch.norm(u)

# Output
# tensor(5.)
```

Un'altra norm comune é la l_1 anche chiamata *Manhattan distance*, questa somma i valori assoluti degli elementi del vettore:

$$\|\mathbf{x}\|_1 = \sum_{i=1}^n |x_i|$$

Questa, rispetto alla l_2 , é meno sensibile ai valori anomali, per calcolarla combiniamo i metodi per il valore assoluto e la somma:

```
torch.abs(u).sum()

# Output
# tensor(7.)
```

Sia la l_2 che la l_1 sono dei casi speciali della norm generica l_p :

$$\|\mathbf{x}\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{\frac{1}{p}}$$

Per quanto riguarda le matrici però le cose si fanno più complicate. Possiamo vedere le matrici sia come insiemi di singoli elementi sia come oggetti che agiscono su vettori trasformandoli in altri vettori. Possiamo chiederci ad esempio quanto il prodotto matrice-vettore $\mathbf{X}\mathbf{v}$ può essere lungo rispetto al vettore $\mathbf{v}$, questo ci porta alla *spectral norm*. Però vediamo la *Frobenius norm* che è più semplice da calcolare ed è definita come la radice quadrata della somma dei quadrati degli elementi della matrice:

$$\|\mathbf{X}\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n x_{ij}^2}$$

Questa si comporta come se fosse una norma l_2 di un vettore a forma di matrice.

Nel deep learning spesso cerchiamo di risolvere problemi di ottimizzazione, massimizzare la probabilità assegnata ai dati osservati, massimizzare il guadagno associato ad un modello di raccomandazione, minimizzare la distanza tra le rappresentazioni delle foto della stessa persona. Queste distanze sono spesso espresse come norme.

2. Calculus

2.1. Derivatives and Differentiation

Molto semplicemente, la derivata é il tasso di variazione di una funzione rispetto alle variazioni dei suoi argomenti. Le derivate ci dicono quanto velocemente una *loss function* cresce o decresce quando incrementiamo o decrementiamo ciascun parametro. Formalmente per delle funzione $f : \mathbb{R} \rightarrow \mathbb{R}$ che mappano scalari in altri scalari, la derivata di f nel punto x é definita come:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

Quando $f'(x)$ esiste, f si dice *differenziabile* in x e quando $f'(x)$ esiste per ogni x in un range di valori, come ad esempio $[a, b]$, diciamo che f é differenziabile su questo set. Non tutte le funzioni sono differenziabili comprese alcune che vorremmo ottimizzare. Tuttavia dato che il calcolo della derivata é fondamentale in quasi tutti gli algoritmi di addestramento, spesso ottimizziamo un surrogato differenziabile.

Possiamo interpretare la derivata $f'(x)$ come il *tasso di variazione istantaneo* di $f(x)$ rispetto ad x . Facciamo degli esempi, definiamo una funzione $u = f(x) = 3x^2 - 4x$:

```
def f(x):  
    return 3 * x ** 2 - 4 * x
```

Impostando $x = 1$ notiamo che $\frac{f(x+h)-f(x)}{h}$ si avvicina a 2 quando h si avvicina a 0. Non abbiamo una dimostrazione matematica rigorosa ma possiamo facilmente notare che $f'(1) = 2$:

```
for h in 10.0*np.arange(-1, -6, -1):  
    print(f'h={h:.5f}, numerical limit={{f(1+h)-f(1))/h:.5f}')
```

Output
h=0.10000, numerical limit=2.30000
h=0.01000, numerical limit=2.03000
h=0.00100, numerical limit=2.00300
h=0.00010, numerical limit=2.00030
h=0.00001, numerical limit=2.00003

Esistono tantissime notazioni per indicare la derivata di una funzione. Data $y = f(x)$ tutte le seguenti espressioni sono equivalenti:

$$f'(x) = y' = \frac{dy}{dx} = \frac{df}{dx} = \frac{d}{dx}f(x) = Df(x) = D_x f(x)$$

I simboli $\frac{d}{dx}$ e D sono *differentiation operators*. Vediamo inoltre alcune derivate di funzioni note:

$$\frac{d}{dx}C = 0 \text{ per ogni costante } C$$

$$\frac{d}{dx}x^n = nx^{n-1} \text{ per } n \neq 0$$

$$\frac{d}{dx}e^x = e^x$$

$$\frac{d}{dx}\ln x = x^{-1}$$

Funzioni composte da funzioni differenziabili sono anch'esse differenziabili. La seguente regola é utile quando lavoriamo con composizioni di qualsiasi funzioni differenziabili f e g e una costante C .

$$\begin{aligned}\frac{d}{dx}[Cf(x)] &= C \frac{d}{dx}f(x) \\ \frac{d}{dx}[f(x) + g(x)] &= \frac{d}{dx}f(x) + \frac{d}{dx}g(x) \\ \frac{d}{dx}[f(x)g(x)] &= f(x) \frac{d}{dx}g(x) + g(x) \frac{d}{dx}f(x) \\ \frac{d}{dx} \frac{f(x)}{g(x)} &= \frac{g(x) \frac{d}{dx}f(x) - f(x) \frac{d}{dx}g(x)}{g^2(x)}\end{aligned}$$

In questo modo possiamo facilmente calcolare la derivata della funzione precedente $3x^2 - 4x$:

$$\frac{d}{dx}[3x^2 - 4x] = 3 \frac{d}{dx}x^2 - 4 \frac{d}{dx}x = 6x - 4$$

Infatti impostando $x = 1$ otteniamo come risultato 2. Da notare che la derivata di una funzione ci indica la sua *pendenza* in un punto.

2.2. Partial Derivatives and Gradients

Per adesso abbiamo calcolato la derivata di funzioni con una sola variabile ma nel deep learning dovremo lavorare con funzioni che hanno diverse variabili. Definiamo $y = f(x_1, x_2, \dots, x_n)$ una funzione con n variabili, la *partial derivative* di y rispetto al suo i -esimo parametro x_i é:

$$\frac{\partial y}{\partial x_i} = \lim_{h \rightarrow 0} \frac{f(x_1, \dots, x_{i-1}, x_i + h, x_{i+1}, \dots, x_n) - f(x_1, \dots, x_i, \dots, x_n)}{h}$$

Per calcolarlo possiamo trattare tutti gli altri parametri come costanti e calcolare la derivata di y rispetto ad x_i . Tutte le seguenti notazioni sono equivalenti:

$$\frac{\partial y}{\partial x_i} = \frac{\partial f}{\partial x_i} = \partial_{x_i} f = \partial_i f = f_{x_i} = f_i = D_i f = D_{x_i} f$$

Possiamo concatenare derivate parziali di una funzione a piú variabili, ognuna rispetto ad una variabile, e ottenere un vettore che chiamiamo **gradiente** della funzione. Supponiamo di avere come input di una funzione $f: \mathbb{R}^n \rightarrow \mathbb{R}$ un vettore n -dimensionale $\mathbf{x} = [x_1, x_2, \dots, x_n]^\top$ e l'output uno scalare. Il gradiente della funzione f rispetto ad $\mathbf{x}$ é un vettore di n derivate parziali:

$$\nabla_{\mathbf{x}} f(\mathbf{x}) = [\partial_{x_1} f(\mathbf{x}), \partial_{x_2} f(\mathbf{x}), \dots, \partial_{x_n} f(\mathbf{x})]^\top$$

Quando il nome delle variabili non crea ambiguitá, possiamo sostituire $\nabla_{\mathbf{x}} f(\mathbf{x})$ con $\nabla f(\mathbf{x})$. Inoltre ci sono le seguenti proprietà:

- $\forall \mathbf{A} \in \mathbb{R}^{m \times n}$ si ha che $\nabla_{\mathbf{x}} \mathbf{A} \mathbf{x} = \mathbf{A}^\top$ e $\nabla_{\mathbf{x}} \mathbf{x}^\top \mathbf{A} = \mathbf{A}$. É l'equivalente della derivata di base, infatti se abbiamo la variabile che moltiplica una costante e deriviamo per quella variabile, possiamo rimuoverla.
- Per matrici quadrate $\mathbf{A} \in \mathbb{R}^{n \times n}$ si ha che $\nabla_{\mathbf{x}} \mathbf{x}^\top \mathbf{A} \mathbf{x} = (\mathbf{A} + \mathbf{A}^\top) \mathbf{x}$ e in particolare $\nabla_{\mathbf{x}} \|\mathbf{x}\|^2 = \nabla_{\mathbf{x}} \mathbf{x}^\top \mathbf{x} = 2\mathbf{x}$

In modo analogo, per ogni matrice $\mathbf{X}$ si ha che $\nabla_{\mathbf{X}} \|\mathbf{X}\|_F^2 = 2\mathbf{X}$

2.3. Chain Rule

Nel deep learning calcolare i gradienti non é semplice dato che spesso si ha a che fare con complesse funzioni innestate. In questi casi si utilizza la *chain rule*. Torniamo per un attimo a funzioni con una sola variabile e supponiamo $y = f(g(x))$ dove $y = f(u)$ e $u = g(x)$ sono entrambe differenziabili. La chain rule ci dice che

$$\frac{dy}{dx} = \frac{dy}{du} \frac{du}{dx}$$

Prendendo quindi una funzione con piú variabili, supponiamo $y = f(\mathbf{u})$ abbia le variabili $u_1, u_2, \dots, u_m$ dove ogni $u_i = g_i(\mathbf{x})$ ha variabili $x_1, x_2, \dots, x_n$ e quindi $\mathbf{u} = g(\mathbf{x})$. Allora la chain rule ci dice che:

$$\frac{\partial y}{\partial x_i} = \frac{\partial y}{\partial u_1} \frac{\partial u_1}{\partial x_i} + \frac{\partial y}{\partial u_2} \frac{\partial u_2}{\partial x_i} + \dots + \frac{\partial y}{\partial u_m} \frac{\partial u_m}{\partial x_i} \text{ e quindi } \nabla_{\mathbf{x}} y = \mathbf{A} \nabla_{\mathbf{u}} y$$

Dove $\mathbf{A} \in \mathbb{R}^{n \times m}$ é una matrice che contiene le derivate del vettore $\mathbf{u}$ rispetto al vettore $\mathbf{x}$. Quindi calcolare il gradiente si può ridurre a calcolare un prodotto vettore-matrice.

3. Automatic Differentiation

Fortunatamente non dobbiamo calcolare noi i gradienti a mano, ormai tutti i più recenti framework di deep learning implementano l'*automatic differentiation* (o *autograd*), ogni volta che i dati passano fra le funzioni il framework costruisce un *computational graph* che traccia le dipendenze fra i valori. Per calcolare le derivate, l'automatic differentiation cammina questo grafo al contrario applicando la chain rule, l'algoritmo che la applica è chiamato *backpropagation*.

Prendiamo come esempio la funzione $y = 2\mathbf{x}^\top \mathbf{x}$ dove appunto $\mathbf{x}$ è un vettore colonna. Iniziamo assegnando ad x un valore iniziale:

```
x = torch.arange(4.0)
x

# Output
# tensor([0., 1., 2., 3.]
```

Però prima di poter calcolare il gradiente di y rispetto ad $\mathbf{x}$ ci serve un posto dove memorizzarlo. Vogliamo inoltre evitare di allocare sempre nuova memoria dato che la derivata va calcolata tantissime volte per un singolo valore. Per farlo utilizziamo:

```
x.requires_grad_(True)
x.grad # Come valore default ha None
```

Adesso possiamo calcolare la nostra funzione e assegnare il risultato ad y :

```
y = 2 * torch.dot(x, x)
y

# Output
# tensor(28., grad_fn=<MulBackward0>)
```

Adesso possiamo calcolare il gradiente di y rispetto ad x chiamando il metodo *backward* per poi accedere all'attributo *grad*:

```
y.backward()
x.grad

# Output
# tensor([ 0.,  4.,  8., 12.]
```

Adesso calcoliamo un'altra funzione di x e il suo gradiente, da notare che PyTorch non pulisce il buffer del gradiente quando ne deve registrare un altro, quello nuovo viene aggiunto al vecchio valore, questo comportamento è utile quando vogliamo ottimizzare funzioni, per resettare il valore però basta chiamare il metodo `x.grad.zero_()`:

```
x.grad.zero_()
y = x.sum()
y.backward()
x.grad

# Output
# tensor([1., 1., 1., 1.]
```

3.1. Backward for Non-Scalar Variables

Quando y è un vettore allora la rappresentazione della derivata di y rispetto ad un altro vettore x è una matrice chiamata *Jacobian* che contiene le derivate parziali di ogni componente di y rispetto ad ogni componente di x , ovviamente per y e x di ordini superiori avremo un tensor di ordine ancora più grande.

Mentre la Jacobiana è utile in tecniche di machine learning più avanzate, nella maggior parte dei casi non ci serve costruirla tutta ma ci interessa ottenere un vettore di gradienti che abbia la stessa forma e dimensione del vettore x . Durante l'addestramento infatti avremo che x rappresenta i pesi della rete e riceviamo in input un batch di dati, la loss function calcolerà su y gli errori per ogni batch, a questo punto non ci interessa calcolare il gradiente per ogni componente rispetto ad ogni altro componente, ovvero come ogni peso si è comportato in ogni foto, o meglio lo calcoliamo ma non ci serve memorizzarlo possiamo sommare tutti i gradienti e capire come quel peso si è comportato in generale in quel batch di dati.

3.2. Detaching Computation

In alcuni casi vogliamo evitare di tracciare il computation graph per alcuni calcoli. Supponiamo di avere $z = x * y$ e $y = x * x$ ma vogliamo concentrarci sull'influenza *diretta* di x su z invece che passare per y . Per farlo possiamo creare una nuova variabile u che assume lo stesso valore di y ma di cui cancelliamo la «provenienza».

```
x.grad.zero_()
y = x * x
u = y.detach()
z = u * x

z.sum().backward()
x.grad == u

# Output
# tensor([True, True, True, True])
```

Otteniamo che il gradiente è uguale u appunto perchè non sappiamo da cosa deriva e viene quindi trattato come costante, se avessimo saputo che $u = x * x$ allora la derivata di x^3 sarebbe stata $3x^2$.

Possiamo comunque calcolare i gradienti di y dato che è u la variabile scollegata:

```
x.grad.zero_()
y.sum().backward()
x.grad == 2 * x

# Output
# tensor([True, True, True, True])
```

4. Linear Neural Networks for Regression

I problemi di *regression* sono quei problemi dove vogliamo fare una previsione di un valore numerico come ad esempio un prezzo. Come esempio prendiamo appunto quello di voler stimare il prezzo di una casa in base alla sua dimensione ed età. Per sviluppare un modello in grado di fare questo abbiamo bisogno di dati, un machine learning il dataset prende il nome di *training set* dove ogni riga si chiama *example* e quello che vogliamo indovinare *label*. Le variabili su cui si basa la previsione prendono il nome di *features*.

4.1. Basics

La *Linear Regression* è uno dei problemi più semplici e popolari nei problemi di regression. La prima assunzione che si fa è che la relazione fra le features $\mathbf{x}$ e la label y sia principalmente lineare, ovvero che il valore atteso $E[Y \mid X = \mathbf{x}]$ può essere espresso come una somma pesata delle features $\mathbf{x}$. Indichiamo con n il numero di examples del nostro dataset, ed usiamo indici in alto per indicare il numero di example o label e indici in basso per numerare le feature di un example. Quindi $\mathbf{x}^i$ indica l' i -esimo example e $x_j^{(i)}$ indica la sua j -esima feature.

4.2. Model

Con l'assunzione fatta prima sulla linearità abbiamo detto che il valore atteso del prezzo può essere espresso come somma pesata delle features:

$$\text{price} = w_{\text{area}} \cdot \text{area} + w_{\text{age}} \cdot \text{age} + b$$

Dove w_{area} e w_{age} sono chiamati *weights* e b *bias*. I pesi determinano l'influenza di ogni feature sulla nostra predizione, il bias determina il valore della predizione quando tutte le features sono a zero. Anche se non è praticamente possibile avere ad esempio una casa con area uguale a 0 abbiamo comunque bisogno di un bias che ci permette di esprimere tutte le funzioni lineari delle nostre features.

Dato un dataset quindi il nostro obiettivo è quello di scegliere i giusti pesi $\mathbf{w}$ e il bias b in modo che, in media, il modello faccia giuste predizioni.

In machine learning lavoreremo spesso con dataset di grandi dimensioni dove conviene utilizzare notazioni algebriche più compatte. Quando il nostro input consiste di d features assegnamo un index a ciascuna ed esprimiamo la nostra predizione $\hat{y}$ come:

$$\hat{y} = w_1 x_1 + \dots + w_d x_d + b$$

Raccogliendo tutte le features in un vettore $\mathbf{x} \in \mathbb{R}^d$ e tutti i pesi in un altro vettore $\mathbf{w} \in \mathbb{R}^d$ possiamo esprimere il modello in modo più compatto:

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b$$

In questo caso il vettore $\mathbf{x}$ corrisponde alle features di un singolo example. Spesso ci conviene fare riferimento alle features dell'intero dataset di n examples tramite la *design matrix* $\mathbf{X} \in \mathbb{R}^{n \times d}$, in quest'ultima ogni riga rappresenta un example e ogni colonna una feature. Per una collezione di features $\mathbf{X}$, la predizione $\hat{\mathbf{y}} \in \mathbb{R}^n$ può essere espressa tramite il prodotto matrice-vettore:

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b$$

Quindi date le features di un training set $\mathbf{X}$ e le corrispondenti labels $\mathbf{y}$, l'obiettivo della linear regression è quello di trovare un vettore di pesi $\mathbf{w}$ e il bias b in modo che, date

delle nuove feature in input, il modello riesca a predire la label con l'errore più basso possibile.

Ovviamente non è possibile che il nostro modello riesca a prevedere ogni valore, ci saranno dataset di n examples dove $y^{(i)}$ non vale esattamente $\mathbf{w}^\top \mathbf{x}^{(i)} + b$ per tutti gli examples. Una principale differenza potrebbe essere ad esempio lo strumento utilizzato per raccogliere i dati.

Adesso, prima di andare a cercare i migliori *parametri* $\mathbf{w}$ e b del modello ci servono ancora due cose:

- Una metrica per misurare la qualità del modello.
- Una procedura per aggiornare il modello e migliorarne la qualità.

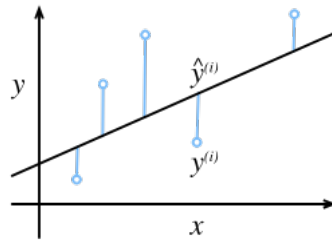
4.3. Loss Function

La *Loss Function* quantifica la distanza tra il valore target reale e quello predetto dal modello. Tipicamente è un numero non-negativo dove i valori piccoli sono a nostro vantaggio e una loss pari a 0 indica una previsione perfetta. Per i problemi di regression la loss function più comune è la *squared error*. Quando la predizione per l'example i è $\hat{y}^{(i)}$ e la corrispondente label reale è $y^{(i)}$ allora la squared error è data:

$$l^{(i)}(\mathbf{w}, b) = \frac{1}{2}(\hat{y}^{(i)} - y^{(i)})^2$$

La divisione $\frac{1}{2}$ non fa realmente differenza ma è comoda dato che poi si va a cancellare quando viene calcolata la derivata.

Vediamo l'adattamento di un modello di linear regression su dati monodimensionali:



Da notare che le grandi differenze fra i valori stimati dal modello e quelli target comportano contributi ancora più grandi alla loss per via della sua forma quadratica, quest'ultima porta sì il modello a evitare errori enormi ma potrebbe anche comportare un'eccessiva sensibilità ai dati anomali.

Per misurare la qualità di un modello sul training set basta calcolare la media della loss su quest'ultimo:

$$L(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n l^{(i)}(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n \frac{1}{2}(\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)})^2$$

Quando addestriamo un modello stiamo cercando i parametri $(\mathbf{w}^*, b^*)$ che minimizzano la loss totale in tutti gli examples di training:

$$\mathbf{w}^*, b^* = \underset{\mathbf{w}, b}{\operatorname{argmin}} L(\mathbf{w}, b)$$

4.4. Analytic Solution

Rispetto a molti altri problemi, la linear regression si può risolvere in modo molto semplice, possiamo infatti trovare i parametri ottimali applicando una semplice formula.

Per prima cosa possiamo includere il bias b nei pesi $\mathbf{w}$ aggiungendo una colonna di tutti 1 alla design matrix. A questo punto il nostro problema diventa quello di minimizzare la formula $\|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2$, finché la design matrix $\mathbf{X}$ ha rango pieno e quindi nessuna feature è linearmente dipendente dalle altre ci sarà un solo punto critico nella loss e corrisponderà al minimo in tutto il dominio della funzione. Prendiamo quindi la derivata della loss rispetto a $\mathbf{w}$ e minimizziamo a 0:

$$\partial_{\mathbf{w}} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 = 2\mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y}) = 0 \text{ quindi } \mathbf{X}^\top\mathbf{y} = \mathbf{X}^\top\mathbf{X}\mathbf{w}$$

Risolvendo per $\mathbf{w}$ la soluzione per il nostro problema di ottimizzazione:

$$\mathbf{w}^* = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}$$

Da notare che questa soluzione è unica quando la matrice $\mathbf{X}^\top\mathbf{X}$ è invertibile ovvero quando le colonne della design matrix sono linearmente indipendenti.

Inoltre una soluzione semplice come questa non dobbiamo aspettarcela per tutti i problemi che ci ritroveremo ad affrontare.

4.5. Minibatch Stochastic Gradient Descent

La miglior tecnica per ottimizzare i parametri di un modello consiste nel ridurre l'errore in step aggiornando i parametri in modo da diminuire sempre di più la loss function, questo algoritmo è chiamato *gradient descent*. L'applicazione più semplice è quella di calcolare la derivata della loss function che non è altro che la media della loss calcolata su ogni example del dataset, questo procedimento però può risultare estremamente lento dato che per ogni cambiamento dovremmo prima iterare sull'intero dataset.

Possiamo provare a considerare l'idea completamente opposta ovvero aggiornare i pesi ad ogni example, l'algoritmo che otteniamo, il *stochastic gradient descent (SGD)* può essere sì buono anche su grandi dataset ma ha dei lati negativi. Il primo, di calcolo, è che i processori sono molto più veloci a fare operazioni piuttosto che muovere dati nelle memorie, è molto più efficiente effettuare una moltiplicazione matrice-vettore piuttosto che tante moltiplicazioni vettore-vettore. Questo significa che è molto più lento processare un example alla volta rispetto a una batch completa. Un altro problema è che alcuni layer del modello, funzionano soltanto quando osserviamo più examples alla volta.

La soluzione sta nel mezzo, invece di prendere una batch intera o un solo example, prendiamo delle *minibatch*. La dimensione di questa dipende da diversi fattori come la quantità di memoria, i layers, la grandezza totale del dataset ecc... Di solito però una potenza di 2 compresa tra 32 e 256 è un buon inizio. Questo ci porta al *minibatch stochastic gradient descent*.

Nella sua forma più semplice, in ogni iterazione t , prendiamo una minibatch casuale $\mathcal{B}_t$ con $|\mathcal{B}|$ examples del training set. Calcoliamo poi la derivata della loss rispetto ai pesi del modello. Infine moltiplichiamo la derivata (gradiente) appena ottenuto per un valore positivo η prestabilito chiamato *learning rate* e sottraiamo il risultato dai parametri attuali. Possiamo esprimere l'aggiornamento in questo modo:

$$(\mathbf{w}, b) \leftarrow (\mathbf{w}, b) - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}_t} \partial_{\mathbf{w}, b} l^{(i)}(\mathbf{w}, b)$$

L'algoritmo esegue quindi:

1. Inizializza i valori dei parametri del modello, di solito in modo casuale.

2. Itera su minibatch casuali del dataset e aggiorna i parametri verso la direzione negativa del gradiente.

Dato che prendiamo una minibatch $\mathcal{B}$ dobbiamo normalizzare per la sua grandezza $|\mathcal{B}|$. Di solito la grandezza della minibatch e il learning-rate sono prestabili dall'utente, questi parametri che non vengono modificati durante il training si chiamano *hyperparameters*. Possono essere comunque ottimizzati attraverso diverse tecniche come la Bayesian optimization. Infine, la qualità del modello ottenuto viene misurata su un set di dati separato e mai visto dal modello chiamato *validation set*.

Dopo aver allenato il modello per un determinato numero di iterazioni, salviamo i parametri del modello che in questo caso denotiamo con $\hat{\mathbf{w}}, \hat{\mathbf{b}}$. Da notare che anche se la nostra funzione è completamente lineare, questi parametri non saranno gli esatti minimizzatori della loss function e non saranno nemmeno deterministici.

In generale il problema principale del deep learning non è trovare parametri ottimali per il dataset di training ma trovare dei parametri ottimali che forniscano buoni risultati su dati mai visti prima, questo è il problema della *generalizzazione*.

4.6. Predictions

Dato il modello $\hat{\mathbf{w}}^\top \mathbf{x} + \hat{\mathbf{b}}$ possiamo iniziare a fare previsioni su nuovi example. Nel deep learning di solito questa fase si chiama *inferenza* ma in realtà è un termine un po' improprio dato l'inferenza si riferisce a una qualsiasi conclusione raggiunta sulla base di prove, inclusi sia i valori dei parametri sia l'etichetta probabile per un'istanza non vista. Nella letteratura statistica l'inferenza fa riferimento all'inferenza dei parametri, per evitare confusione la chiameremo «previsione».

4.7. Vectorization for Speed

Quando addestriamo il modello vorremmo processare minibatches in modo simultaneo, per farlo abbiamo bisogno di vettorizzare i calcoli e sfruttare librerie di algebra lineare veloci invece di scrivere dei semplici loop in Python. Per vedere l'impatto di questo consideriamo due metodi per sommare vettori, per prima cosa inizializziamo due vettori da 10.000 elementi, come primo metodo facciamo un loop sui vettori.

```
n = 10000
a = torch.ones(n)
b = torch.ones(n)
```

Adesso possiamo verificare i tempi andando a fare la somma con il primo metodo:

```
c = torch.zeros(n)
t = time.time()
for i in range(n):
    c[i] = a[i] + b[i]
f'{time.time() - t:.5f} sec'

# Output
# '0.17802 sec'
```

Se invece utilizziamo l'operatore `+`:

```
t = time.time()
```



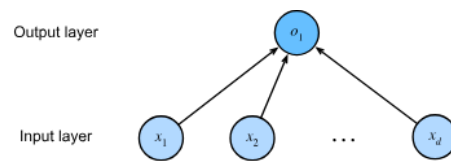
```
d = a + b
f'{time.time() - t:.5f} sec'

# Output
# '0.00036 sec'
```

Il secondo metodo è estremamente più veloce, questo infatti somma uno ad uno gli elementi in modo parallelo.

4.8. Linear Regression as a Neural Network

Le reti neurali sono abbastanza potenti da comprendere i modelli lineari e rappresentarli come reti dove ogni feature è rappresentata da un neurone di ingresso e sono tutti collegati in modo diretto all'output. Quindi:



Gli input sono $x_1, \dots, x_d$, ci riferiamo a d come numero di inputs oppure la dimensione delle features nel layer di input, l'output è o_1 ed è uno soltanto dato che stiamo provando a prevedere un solo valore. Dato che il neurone calcolato è soltanto uno e gli altri sono tutti dati possiamo pensare alla linear regression come una fully connected linear network.

5. Generalization

Per capire meglio questo concetto immaginiamo due studentesse che devono prepararsi per un esame, Ellie ha una memoria incredibile e riesce a ricordare praticamente ogni domanda di ogni esame passato ma questo significa che potrebbe comunque bloccarsi su una nuova domanda mai vista prima. Irene invece non riesce a ricordare nulla ma è bravissima a trovare schemi ricorrenti. Se l'esame consistesse veramente in solo domande riciclate allora Ellie supererebbe facilmente Irene, infatti anche se quest'ultima ottenesse un buonissimo 90% andrebbe comunque a perdere contro il 100% di Ellie, se però l'esame consistesse interamente in domande nuove allora Irene manterrebbe comunque il suo punteggio del 90% mentre Ellie fallirebbe completamente.

Il nostro obiettivo è quello di trovare dei pattern, ma come siamo sicuri di averne trovato uno invece di aver memorizzato tutti i dati di addestramento? Non abbiamo bisogno di prevedere i prezzi delle azioni di ieri o riconoscere malattie già diagnosticate, dobbiamo trovare pattern in situazioni mai viste prima. Questo problema della generalizzazione è il problema fondamentale del machine learning e di tutta la statistica.

Il fenomeno in cui l'adattamento risulta più vicino al training set che alla distribuzione sottostante è chiamato *overfitting* e le tecniche per contrastarlo sono chiamate *regularization methods*.

5.1. Training Error and Generalization Error

In un ambiente standard assumiamo che training set e test set siano presi in modo indipendente dalla stessa distribuzione, questa di solito viene chiamata *IID Assumption*. Da notare che senza questa assunzione non andremmo da nessuna parte, infatti per quale motivo dei dati presi da una distribuzione $P(X, Y)$ dovrebbero dirci come fare previsioni su dati generati da una diversa distribuzione $Q(X, Y)$? Per fare questi passaggi è necessario formulare ipotesi solide su come le due distribuzioni siano collegate. Prima però basiamoci sul caso in cui sono uguali ovvero nella IID.

Iniziamo distinguendo il *training error* R_{emp} che è calcolato sul training set e il *generalization error* R che invece è quello che ci aspettiamo dalla distribuzione. Possiamo vedere il generalization error come quello che vedremmo se applichiamo il nostro modello ad una sequenza infinita di dati presi dalla stessa distribuzione. Formalmente il training error è espresso come una somma:

$$R_{\text{emp}}[\mathbf{X}, \mathbf{y}, f] = \frac{1}{n} \sum_{i=1}^n l(\mathbf{x}^{(i)}, y^{(i)}, f(\mathbf{x}^{(i)}))$$

Mentre il generalization error è espresso come un integrale:

$$R[p, f] = E_{(\mathbf{x}, y) \sim P}[l(\mathbf{x}, y, f(\mathbf{x}))] = \int \int l(\mathbf{x}, y, f(\mathbf{x})) p(\mathbf{x}, y) d\mathbf{x} dy$$

Non possiamo mai calcolare esattamente il generalization error R , nessuno infatti ci dirà mai la forma precisa della funzione di densità $p(\mathbf{x}, y)$. Inoltre, non possiamo esaminare una sequenza infinita di dati. Nella pratica quindi dobbiamo fare una stima del generalization error applicando il nostro modello su un test set indipendente su una collezione di esempi $\mathbf{X}'$ e di labels $\mathbf{y}'$ che non sono presenti nel dataset.

5.2. Model Complexity

Di solito quando lavoriamo con tanti dati, il training e il generalization error tendono ad essere simili, quando però lavoriamo con modelli più complessi o con pochi examples ci aspettiamo che il training error scenda ma il generalization error sia abbastanza distante o che salga e questo non ci deve sorprendere. Immaginiamo un modello che, per ogni dataset di n esempi, riesce a trovare un set di parametri che si adattano perfettamente alle labels anche se assegnate in modo randomico, in questo caso anche se il modello si comporta benissimo nel training come possiamo capire qualcosa sul generalization error? Per quel che sappiamo il generalization error potrebbe anche essere peggiore di sparare a caso.

In generale, in assenza di restrizioni sul modello, basandoci soltanto su come il modello si è adattato al training set non possiamo concludere che abbia trovato un pattern generalizzabile.

Quando un modello è in grado di riconoscere delle labels, un basso training error non implica obbligatoriamente anche un basso generalization error ma nemmeno un alto generalization error. Le reti neurali profonde non ci permettono di trarre conclusioni basandoci esclusivamente sull'errore di addestramento, in questi casi dobbiamo fare maggiore affidamento sull'errore di validazioni ovvero quello calcolato su altri examples rispetto al training.

5.3. Underfitting or Overfitting

Quando compariamo training e validation error dobbiamo stare attenti a due situazioni particolari. Per prima cosa vogliamo vedere se i due errori sono abbastanza grandi ma c'è una leggera differenza fra i due. Se il nostro modello non riesce a ridurre il training error allora il modello è troppo semplice per riuscire a individuare un pattern, inoltre, dato che il gap tra training e generalization error è piccolo significa che possiamo usare un modello più complesso per ottenere risultati migliori. Questo fenomeno prende il nome di **underfitting**.

Per seconda cosa vogliamo controllare se il training error è decisamente inferiore del validation error, questo indica **overfitting**. Da notare che l'overfitting non è sempre una brutta cosa. Soprattutto nel deep learning, i migliori modelli ottengono spesso risultati superiori nel training rispetto alla validazione. Il nostro obiettivo finale è solamente che l'errore di generalizzazione sia basso, se poi c'è un gap con l'errore di addestramento non ci interessa, a meno che questo gap non sia talmente alto da non permettere una discesa dell'errore di validazione. Da notare che se l'errore di addestramento è zero allora il gap di generalizzazione è esattamente uguale all'errore di generalizzazione e quindi l'unico modo per ridurlo è ridurre il gap attraverso tecniche specifiche.

5.4. Model Selection

Di solito scegliamo il nostro modello finale soltanto dopo aver valutato diversi modelli, questo passo è chiamato appunto *model selection*.

Non dovremmo toccare il nostro set di test finché non abbiamo scelto gli hyperparameters, infatti se dovessimo usare il test set durante la scelta del modello rischiamo di andare in overfitting. Infatti andare in overfitting nel training non è un problema dato

che abbiamo poi la valutazione che ci garantisce affidabilità, ma se overfittiamo i dati di test non ce ne accorgeremmo.

Non dobbiamo quindi mai basarci sui dati di test per la scelta del modello, ma non possiamo nemmeno affidarci soltanto a quelli di training perché non siamo in grado di stimare l'errore di generalizzazione su questi. Nelle applicazioni pratiche la questione è ancora più confusa, infatti dovremmo utilizzare i dati di test una sola volta per valutare il modello migliore o per confrontare un numero limitato di modelli ma nella realtà i dati di test raramente vengono scartati dopo un solo utilizzo.

La prassi è quella di dividere il dataset in tre parti, includendo un set di validazione oltre al set di addestramento e di test. Il risultato è che la differenza tra set test e validation è molto ambigua, per questo spesso si utilizza soltanto training e validation.

5.5. Cross-Validation

Quando i dati per l'addestramento sono pochi non siamo in grado di separare abbastanza dati per creare un buon validation set. Una soluzione comune è il K – fold cross-validation dove il training set viene diviso in K sottoinsiemi con nessun elemento in comune, poi l'addestramento e la valutazione vengono effettuati K volte, ogni volta addestrando su $K - 1$ sottoinsiemi di training e valutando su l'ultimo sottoinsieme che viene preso come validation. Infine, l'errore di training e validation sono stimati facendo la media dei K risultati.

6. Weight Decay

Dato che abbiamo introdotto il problema dell'overfitting, possiamo introdurre la prima *regularization technique*. Ricordiamo però che possiamo sempre ridurre l'overfitting aggiungendo più dati di addestramento ma ovviamente potrebbe richiedere tanto tempo o spesa oppure potrebbe essere impossibile in determinati casi. Assumiamo quindi di avere già abbastanza dati e di qualità.

Una prima tecnica è quella di limitare il numero di feature, tuttavia limitarsi a scartare feature è un metodo troppo grossolano.

6.1. Norms and Weight Decay

Invece di manipolare il numero di parametri, il weight decay si basa sul restringere i valori che i parametri possono prendere. Viene anche chiamata l_2 regularization fuori dal mondo del deep learning. La tecnica si basa sul fatto che fra tutte le funzioni f , la funzione $f = 0$ è la più semplice, possiamo quindi misurare la complessità di una funzione in base alla distanza dei suoi parametri da zero. Ma quanto precisi dobbiamo essere mentre misuriamo questa distanza? Non c'è una risposta giusta.

Una semplice interpretazione potrebbe essere quella di misurare la complessità di una funzione lineare $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}$ con qualche norma dei suoi pesi, ad esempio $\|\mathbf{w}\|^2$. Il metodo più comune per garantire che il vettore dei pesi sia di piccole dimensioni consiste nell'aggiungere la sua norma come termine di penalizzazione al problema di minimizzazione della loss. Il nostro obiettivo quindi non è più quello di minimizzare la prediction loss sulle training labels ma minimizzare la somma della prediction loss e il penalty term. A questo punto se il vettore dei pesi cresce troppo, l'algoritmo di apprendimento potrebbe concentrarsi sul minimizzare la norma dei pesi piuttosto che minimizzare l'errore di training e questo è esattamente quello che vogliamo. Significa infatti che l'algoritmo si trova costretto a fare un compromesso:

- Se tenta di abbassare troppo l'errore nel training ingrandendo i pesi allora la norma diventa alta.
- L'algoritmo allora preferirà accettare un errore di training leggermente più alto pur di mantenere i pesi piccoli e contenuti. La loss originariamente l'avevamo espressa come:

$$L(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} (\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)})^2$$

Dove ricordiamo che: $\mathbf{x}^{(i)}$ sono le features, $y^{(i)}$ sono le label di ogni example i e $(\mathbf{w}, b)$ sono i pesi e il bias. Per penalizzare il vettore dei pesi dobbiamo aggiungere $\|\mathbf{w}\|^2$ alla loss, ma come fa il modello a bilanciare la loss standard con questa nuova penalità? Lo facciamo tramite la *regularization constant* λ , un hyperparameter non negativo il cui valore ottimale viene scelto valutando le prestazioni sul set di validazione. La nuova funzione diventa quindi:

$$L(\mathbf{w}, b) + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

In questo modo otteniamo:

- Se $\lambda = 0$ la penale si azzerà e si torna alla funzione di loss originale.

- Se $\lambda > 0$ stiamo imponendo un vincolo che forza la magnitudo di $\|\mathbf{w}\|$ a rimanere contenuta, più grande è λ e più severa sarà la penalità per pesi grandi. Si eleva al quadrato sempre per semplicità algebrica per il calcolo delle derivate.

Perché si sceglie la norma l_2 , e non ad esempio la l_1 ? Differenze comportamentali:

- La norma l_2 eleva al quadrato ogni singolo peso w_j , di conseguenza assegna una penalità sproporzionalmente alta ai pesi di grande valore. In questo modo spinge l'algoritmo ad evitare singoli pesi giganti e a distribuire il carico in modo uniforme. Inoltre rende il modello più robusto, se una singola variabile in input contiene rumore o un errore di misurazione, il suo impatto sul risultato sarà limitato perché il peso associato a questa è piccolo.
- La norma l_1 penalizza i pesi in modo proporzionale al loro valore assoluto senza accentuare i valori grandi. Questo porta i pesi ad azzerarsi completamente e il modello concentra tutta la sua capacità solo su un piccolo sottoinsieme di feature. Il vantaggio è che funziona come un meccanismo automatico di selezione delle variabili, se un peso diventa zero allora quella specifica feature non serve più per fare la predizione e questo permette di risparmiare memoria, calcolo e risorse nella fase di raccolta, archiviazione o trasmissione dei dati futuri.

La formula con minibatch stochastic gradient descent con regolarizzazione l_2 diventa quindi:

$$\mathbf{w} \leftarrow (1 - \eta\lambda)\mathbf{w} - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \mathbf{x}^{(i)} (\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)})$$

Notiamo che i termini dell'equazione sono:

- $(1 - \eta\lambda)\mathbf{w}$: sia η che λ sono numeri piccoli e positivi e quindi questo termine è un numero leggermente inferiore a 1, ad ogni passo di addestramento il peso viene prima «ristretto» ovvero moltiplicato per questo numero.
- Il resto è la classica correzione basata sull'errore commesso dal modello.

Si chiama weight decay perché se azzeriamo l'errore del modello sui dati allora l'aggiornamento dei pesi sarebbe soltanto

$$\mathbf{w} \leftarrow (1 - \eta\lambda)\mathbf{w}$$

E questo significa che i pesi decadrebbero esponenzialmente verso lo zero ad ogni iterazione.

Notiamo che per il bias b non viene fatta nessuna penalizzazione, questo perché sono i pesi che controllano la pendenza e la reattività delle singole variabili di input e causare quindi overfitting se troppo grandi, il bias invece sposta semplicemente la funzione in alto o in basso e non aggiunge oscillazioni al modello.

Infine notiamo che la l_2 regularization e il weight decay potrebbero non essere perfettamente equivalenti per altri algoritmi di ottimizzazione.

- Nella SGD aggiungere $\|\mathbf{w}\|^2$ alla loss o moltiplicare i pesi per $(1 - \eta\lambda)$ produce la stessa identica operazione matematica.
- Con ottimizzatori più complessi però questo non è vero, ad esempio con *Adam*. Infatti esistono varianti come *AdamW* che utilizzano anche il weight decay.

7. Linear Neural Network for Classification